

MBR HTML Generator

User Guide

A comprehensive walkthrough of the WordPress plugin, with particular focus on the Classify pipeline and the Custom CSS drawer.

Version 1.7.1

Made by Robert · madebyrobert.co.uk

Little Web Shack · littlewebshack.com

Contents

1. What this plugin does
2. The interface at a glance
3. Composing content in the editor
4. The output panel toolbar
5. Tidy — semantic cleanup
6. Classify — inline styles to utility classes
7. The Custom CSS drawer
8. Selecting elements with CSS
9. Worked example: a styled hero section
10. Worked example: refactoring a TinyMCE document with Classify
11. Combining Tidy, Classify and Custom CSS
12. Tips, tricks and idioms
13. Privacy and local storage
14. The frontend shortcode
15. Troubleshooting

1. What this plugin does

MBR HTML Generator is a small, focused tool for composing HTML in a friendly visual editor and getting clean, copy-paste-ready markup out the other side. It runs entirely in your browser: there is no server round-trip, no telemetry, and nothing is saved to your WordPress database beyond the plugin's own settings.

The typical workflow is: type, format and style your content in the editor on the left, watch the formatted HTML appear in the panel on the right, and copy or download the result when you are happy with it. Three pipeline stages shape what comes out: Tidy (semantic cleanup of the editor's raw HTML), Classify (extracting inline styles into reusable CSS classes) and the Custom CSS drawer (your own rules, applied live in the editor and included in the output). The three are independent toggles and can be used in any combination.

The plugin is also available as a shortcode, `[mbr_html_generator]`, which exposes the same tool to logged-out visitors on a public page. The shortcode and the admin tool share their interface and their local-storage state per browser, so settings you change in one are visible in the other.

Where to find it

In the WordPress admin, the tool lives under **Tools** → **HTML Generator**. On a public page or post, drop in the `[mbr_html_generator]` shortcode and the same interface renders inline.

Browser support

Any modern evergreen browser will run it: recent Chrome, Edge, Firefox or Safari. The plugin uses standard DOM APIs (DOMParser, localStorage), TinyMCE for the editor, and js-beautify for HTML formatting. It works on tablets and large phones, although the side-by-side layout collapses to a stack on narrow screens.

2. The interface at a glance

The window is split into two panels, with a header bar across the top and a stats footer at the bottom.

Editor panel (left)

Houses TinyMCE, the visual editor. Compose here exactly as you would in any WordPress post: headings, paragraphs, lists, images, tables, links, the lot. The editor's panel header has two buttons: CSS (opens the Custom CSS drawer, covered in chapter 7) and Clear (empties the editor; the persisted content is removed at the same time).

Below the editor sits the CSS drawer. It is collapsed by default and only appears when you click the CSS button; when open it pushes the editor up to make room and gives you a monospace textarea for writing styles.

Output panel (right)

Shows the live HTML produced by the editor. The panel header has a row of toggle buttons (Tidy, Classify, Format, Wrap) and two action buttons (Download, Copy). The numbers update as you type.

Header bar (top)

Holds the page title and the theme toggle. The theme switches between light and dark modes; both the editor and the output panel restyle together, and your choice is remembered across sessions.

Footer (bottom)

Three running stats: character count, word count and line count of the current output. Updated live as you compose and as toggles change the output.

3. Composing content in the editor

The editor is the standard TinyMCE rich-text editor with a small set of practical tweaks to suit how the tool is used. You can type prose, apply formatting, drop in images by URL, build tables, paste from Word or Google Docs, and so on. The important things to know about its behaviour:

- **Images are URL-only.** The Image dialog accepts a URL but does not allow file uploads. This is deliberate: with no server endpoint to receive uploads, the only fallback would be to base64-encode the image into the `src` attribute, which produces hundreds of kilobytes of text per photo and unusable output. Paste a URL instead, or upload to your media library first.
- **Source code view.** The `</>` button on the toolbar (and the View → Source code menu item) opens a modal showing the raw HTML inside the editor. Edit it directly and click Save to push the changes back. This is how you assign classes to elements when working with the Custom CSS drawer.
- **Drag-drop is restricted.** Dropping an image file onto the editor is blocked for the same reason as upload: it would inline the file as a data URI. Drop URLs or text instead, or use the Image button in the toolbar.
- **Quick-toolbars.** Selecting text shows a small floating toolbar with bold, italic, link and heading shortcuts. Clicking into an empty paragraph shows a Quick Table inserter.

Your editor content is auto-saved to your browser's local storage approximately half a second after you stop typing. Refreshing the page restores exactly where you left off; clicking Clear removes both the editor content and the saved copy.

The Format dropdown

The block-level format dropdown (where it says Paragraph by default) lets you switch between paragraph, headings H1-H6, blockquote, and preformatted blocks. Heading levels matter for accessibility and SEO; do not skip levels (H1 then H3 with no H2 in between) and do not use them just for size.

Inline formatting

Bold, italic, underline and strikethrough work the way you would expect. Note that TinyMCE may use `<strong>` and `<em>` rather than `<b>` and `<i>`; the former are semantically richer and screen readers treat them with appropriate emphasis. If you need the visual tag specifically, switch to source code view and edit it directly.

4. The output panel toolbar

Six controls in the panel header shape how the output looks and what you do with it. Each remembers its on/off state per browser, so once you have your defaults set the tool comes back exactly as you left it.

Control	Default	What it does
Tidy	On	Cleans up TinyMCE's raw output: unwraps standalone images from paragraphs, removes redundant spans wrapping headings and paragraphs, drops empty paragraphs, merges non-conflicting inline styles onto block elements. Detailed in chapter 5.
Classify	Off	Extracts inline style attributes into reusable, atomic CSS utility classes and emits the rules as a <code><style></code> block at the top of the output. Detailed in chapter 6.
Format	On	Pretty-prints the HTML with line breaks and indentation. Toggle off for single-line minified output.
Wrap	On	Soft-wraps long lines in the output panel display only; does not change the actual characters in the output. Toggle off when you want to see exactly where long lines live.
Download	—	Saves the current output to your downloads folder as a .html file. The file's content reflects all active toggles and the Custom CSS drawer.
Copy	—	Copies the current output to your clipboard. Same content as Download. The button flashes briefly to confirm the copy succeeded.

Tip. The output is regenerated from the editor's current contents every time any toggle changes. There is no separate 'Apply' step. Whatever you can see in the output panel is exactly what Copy and Download will give you.

5. Tidy — semantic cleanup

TinyMCE produces HTML that is functional but, in places, more verbose than it needs to be. Headings get wrapped in spans that carry the heading's colour and size as inline styles, even though those styles could sit on the heading itself. Images that are not aligned to text get wrapped in their own paragraph. Empty paragraphs sneak in between blocks. Tidy is a pass that resolves these patterns, leaving cleaner, more semantic markup.

What Tidy actually does

- **Unwraps standalone images from paragraphs.** A `<p>` containing nothing but an `<img>` (and optional whitespace) becomes the image directly. `<figure>` wrappers are left alone.
- **Removes redundant spans on headings and paragraphs.** When a heading or paragraph has a single child `<span>` carrying inline styles, the styles are merged onto the block and the span is unwrapped, provided this does not change the cascade for any conflicting properties.
- **Drops empty paragraphs.** Paragraphs containing only whitespace, ` `, or `<br>` are removed, since they almost always represent a stray Enter rather than an intentional blank line. Use spacing CSS for vertical rhythm instead.
- **Strips empty style attributes.** An `style=""` that survived editing is removed.

When to keep Tidy on

Almost always. The cleanup is conservative — it never changes the rendered output, only the structure of the HTML — and the result is consistently more semantic. The main reason to turn it off is if you want to see precisely what TinyMCE produced before any post-processing, which is useful for debugging or for understanding exactly which markup pattern caused some downstream issue.

Tidy is the recommended companion to Classify. With Tidy off, Classify sees the spans-on-headings pattern and generates atomic classes for the spans rather than for the headings, producing markup that is technically correct but uglier. With Tidy on first, the styles are already where you would want them and Classify's output is cleaner.

Example

Input from TinyMCE:

```
<h1><span style="color: #e03e2c; font-size: 36px;">Welcome</span></h1>
<p></p>
<p>&nbsp;</p>
<p>Some prose follows.</p>
```

With Tidy on:

```
<h1 style="color: #e03e2c; font-size: 36px;">Welcome</h1>  
  
<p>Some prose follows.</p>
```

6. Classify — inline styles to utility classes

Classify is the most opinionated of the three pipeline stages. With it on, every inline style attribute in the editor's HTML is extracted into a reusable, atomic CSS utility class — one class per property/value pair — and the generated rules are emitted as a single `<style>` block at the top of the output. Identical declarations across elements share the same class, so a document with five elements all styled `color: red` ends up with one `.c-red` rule shared between them.

If you are familiar with Tailwind CSS, the result will look immediately familiar. If you are not: think of it as turning a soup of repeated inline styles into a lean stylesheet where every visual decision lives in exactly one place.

The simplest possible example

Input:

```
<h1 style="color: #e03e2c; font-size: 36px;">Welcome</h1>
<p style="color: #e03e2c;">Same red as the heading.</p>
```

Output with Classify on:

```
<style>
.c-e03e2c { color: #e03e2c; }
.fs-36 { font-size: 36px; }
</style>
<h1 class="c-e03e2c fs-36">Welcome</h1>
<p class="c-e03e2c">Same red as the heading.</p>
```

The heading's two inline declarations became two atomic classes. The paragraph's single declaration reused the existing `.c-e03e2c` class rather than producing a duplicate rule. Three elements, two rules — and if you add a third red element later, no new rule is added at all.

How class names are generated

Names follow the pattern `{property-abbreviation}-{value-token}`, kebab-cased. Common CSS properties have short abbreviations baked into the plugin (the full list is on the next page); unknown properties fall back to the full kebab-cased property name. Values are normalised for class-name safety:

- Hex colours: the leading `#` is stripped. `#e03e2c` becomes `e03e2c`.
- Pixels are the implicit default unit and are stripped. `36px` becomes `36`.
- Decimals become `p`. `1.5em` becomes `1p5em` (the dot would otherwise be parsed as a class selector).
- Percent becomes `pc`. `100%` becomes `100pc`.

- Negative numbers become n followed by the magnitude. -10px becomes n10.
- Spaces in multi-token values become hyphens. 10px 20px becomes 10-20 (giving .m-10-20).
- Everything is lower-cased. #4E CDC4 becomes 4ecdc4 in the class name even though the original case is preserved in the CSS rule itself.

Property abbreviation reference

Commonly-used properties have short abbreviations. Anything not in this list falls back to the full kebab name (so `caret-color: red` becomes `.caret-color-red`).

Property	Abbrev.	Property	Abbrev.
color	c	border-radius	rad
background-color	bg	border-color	bdc
background-image	bgi	border-style	bds
background-position	bgp	border-width	bdw
background-repeat	bgr	width	w
background-size	bgs	height	h
font-size	fs	max-width	mw
font-weight	fw	min-width	mnw
font-family	ff	max-height	mh
font-style	fst	min-height	mnh
text-align	ta	display	d
text-decoration	td	position	pos
text-transform	tt	opacity	op
text-indent	ti	z-index	z
line-height	lh	overflow	ov
letter-spacing	lsp	cursor	cur
white-space	ws	float	fl
margin	m	clear	clr
margin-top	mt	vertical-align	va
margin-right	mr	box-shadow	bsh
margin-bottom	mb	text-shadow	tsh
margin-left	ml	transform	tf
padding	p	transition	trs
padding-top	pt	border	b
padding-right	pr	border-top	bt
padding-bottom	pb	border-right	brt
padding-left	pl	border-bottom	bb
		border-left	bl

Fallback for un-tokenisable values

Some CSS values contain characters that cannot live safely in a class name: commas, parentheses, quotes, slashes. These show up in functional notation like `rgb()`, `rgba()`, `url()`, `calc()`, `var()`, and in quoted font-family lists. When Classify encounters such a value it can still emit a rule for it — the CSS itself is perfectly valid — but it cannot derive a readable class name from it. The fallback is a per-property sequential counter:

```
<p style="color: rgba(255, 0, 0, 0.8); font-family: Arial, sans-serif;">x</p>
```

```
/* becomes */
```

```
<style>
.c-1 { color: rgba(255, 0, 0, 0.8); }
.ff-1 { font-family: Arial, sans-serif; }
</style>
<p class="c-1 ff-1">x</p>
```

The counter resets to 1 per property, so `.c-1`, `.c-2` and so on are distinct from `.bg-1` and `.ff-1`. Within a single document, the same value still deduplicates to the same counter class — a second `rgba(255, 0, 0, 0.8)` elsewhere in the document reuses `.c-1` rather than getting a new number.

Existing classes are preserved

If you already have a class attribute on an element, Classify appends to it rather than replacing it:

```
<h1 class="hero" style="color: red;">x</h1>
```

/ becomes */*

```
<h1 class="hero c-red">x</h1>
```

The existing hero class is left exactly where it was; the generated c-red is added after it. Duplicate class names are skipped, so if your existing markup happened to contain class="c-red" already, Classify will not add a second copy of it.

Pairing Tidy with Classify

Classify works on its own, but it produces materially cleaner output when Tidy runs first. The reason: TinyMCE writes styled headings as `<h1><span style="color: red; font-size: 36px;">x</span></h1>` — a span carrying the styles inside the heading. With Tidy off, Classify sees the span and the heading separately:

```
<h1><span class="c-red fs-36">x</span></h1>
```

Functionally fine, but the span is now redundant. With Tidy on first, the span is unwrapped onto the heading and Classify sees the consolidated markup:

```
<h1 class="c-red fs-36">x</h1>
```

Tidy is on by default for exactly this reason. There is no harm in leaving it on all the time.

When you should not use Classify

- **Outputting into a system that already styles every class.** If your destination CMS or design system already has classes called c-red, fs-36 and the like, the generated rules will collide. Either rename the conflict in your codebase or live with inline styles.
- **Outputting into Gutenberg/block editor.** Gutenberg has its own class system and does not always play nicely with foreign class names. For Gutenberg, inline styles often paste back in more reliably than classes.
- **One-off content with very few inline styles.** If your document has three inline styles, three atomic classes plus a four-line `<style>` block is bigger than the inline styles it replaces. Classify earns its keep on documents with repeated styles.

7. The Custom CSS drawer

The Custom CSS drawer lets you write your own CSS rules that are both applied live in the editor preview and included verbatim at the top of the generated output. It turns the tool from a pure HTML generator into a small playground for composing styled content with feedback on every keystroke.

Opening and closing

Click the CSS button in the editor's panel header (the one with the palette icon, next to Clear). The drawer slides up from below the editor, the editor itself shrinks to make room, and the textarea inside the drawer takes focus so you can start typing immediately.

To close, click the CSS button again, or use the Hide button on the drawer header (chevron icon). The drawer's open/closed state is remembered across sessions, so if you leave it open and come back tomorrow, it will still be open.

Writing rules

The textarea is a plain monospace text input — no syntax highlighting, no auto-complete, no linting. That keeps the plugin lean and the textarea responsive. Write whatever CSS you like, in whatever style you like:

```
/* Tag selectors apply to every matching element */
h1 {
  font-family: Georgia, serif;
  letter-spacing: -0.02em;
}

/* Attribute selectors are handy for images */
img[alt="Hero"] {
  border-radius: 16px;
  box-shadow: 0 8px 24px rgba(0, 0, 0, 0.1);
}

/* Class selectors need a class on the element; see chapter 8 */
.card {
  padding: 20px;
  background: #f7f9fc;
  border-radius: 12px;
}
```

Live preview

As you type, the editor on the left updates roughly 150 milliseconds after you stop. Internally, the plugin injects a `<style id="mbr-custom-css">` element into TinyMCE's content iframe and rewrites its contents on every change. The same CSS you can see in the drawer is what the editor is using to render.

Two consequences worth knowing. First, what you see in the editor is exactly what the output will look like when those styles are loaded on the receiving page — there is no preview/production drift. Second, the editor has its own baseline styles for usability (a comfortable body padding, a default font) which TinyMCE applies via its `content_style` configuration. Your CSS sits alongside those baselines and follows the normal CSS cascade. If something refuses to take effect, the most common reason is that one of TinyMCE's defaults is winning on specificity; adding `!important` or using a more specific selector usually resolves it.

Drawer header controls

- **Lines counter.** Shows the current line count of the drawer's contents, updated as you type. Empty drawer shows 0.
- **Clear.** Empties the drawer in one click. There is no confirmation prompt and your browser's undo (Ctrl-Z) on the textarea will typically not bring the content back, so use deliberately. This is intentionally separate from the `Clear` button in the editor's panel header: clearing the editor leaves your CSS untouched, and vice versa.
- **Hide.** Collapses the drawer without changing its contents. Reopens to exactly the same state.

How the drawer affects the output

Whenever the drawer has non-empty content, your CSS appears in a `<style>` block at the top of the output. What happens next depends on whether Classify is also on:

Classify off. Your CSS gets its own `<style>` block, sitting before the rest of the HTML:

```
<style>
.card { padding: 20px; }
</style>
<div class="card">Some content</div>
```

Classify on. Your CSS and the generated atomic rules merge into the same `<style>` block, with your CSS first:

```
<style>
.card { padding: 20px; }
.c-e03e2c { color: #e03e2c; }
.fs-36 { font-size: 36px; }
</style>
<h1 class="c-e03e2c fs-36">Welcome</h1>
<div class="card">Card</div>
```

The ordering — your CSS first, atomic classes second — is deliberate. It matches utility-class cascade semantics: your CSS acts as the base layer, the atomic classes are utilities that can override the base by source order. So a rule like `.c-red { color: blue }` in your CSS would be overridden by Classify's auto-generated `.c-red { color: red }` rule that follows. This is how utility-class systems typically behave and is usually what you want.

Independence

The drawer is completely independent of Tidy and Classify. Non-empty drawer means your CSS appears in the output; empty drawer means it does not. There is no master toggle for 'include my CSS' — the presence of content is the toggle. Less switching, less to remember.

Cross-tab sync

If you have the tool open in two tabs at once and edit the CSS in one, the other picks up the change automatically. This is via the browser's storage event and happens within milliseconds. The same applies to the editor content.

What gets persisted

The drawer's contents are saved to your browser's local storage approximately 150 milliseconds after you stop typing. The open/closed state is saved when you toggle it. Both survive page refreshes, browser restarts, and operating-system reboots. They are stored per-browser; clearing site data or switching to a different browser starts you with an empty drawer. None of it leaves your machine.

What is not validated

The plugin does not inspect, lint, or sanitise your CSS. If you write something syntactically broken, the broken bit will sit in the output as-is and may prevent any subsequent rules from taking effect (CSS parsers stop at the first broken declaration in a rule, but typically continue past a broken rule). The feedback loop is the live preview: if a rule fails to apply to the editor, the browser has either ignored it or applied it but lost to specificity.

8. Selecting elements with CSS

The Custom CSS drawer is only as useful as the selectors you can write. Four kinds of selector cover most realistic use cases:

Tag selectors — broadest, easiest

Target every element of a given type. The simplest possible thing that works:

```
h1 { font-family: Georgia, serif; }
p { line-height: 1.7; }
blockquote {
  border-left: 4px solid #2a5cad;
  padding-left: 16px;
  font-style: italic;
}
img { border-radius: 8px; }
ul li { margin-bottom: 4px; }
```

Tag selectors are the obvious choice when you want consistent treatment across an entire document type. They do not require you to touch the markup at all — the editor's existing tags are the targets.

Attribute selectors — specific without touching markup

Target elements based on the value of an attribute. Particularly useful for images, where you might want different treatment for the hero shot versus other images on the page:

```
/* Match images whose src contains "hero" (filename or path fragment) */
img[src*="hero"] {
  border-radius: 16px;
  box-shadow: 0 8px 24px rgba(0, 0, 0, 0.1);
}

/* Match images by their alt text */
img[alt="Author photo"] {
  border-radius: 50%;
  width: 120px;
}

/* Match links opening in a new tab */
a[target="_blank"]::after {
  content: " ↗";
  color: #2a5cad;
}
```

The exact-match form is `[attr="value"]`; the substring form is `[attr*="value"]`. The substring form is more forgiving with filenames — `img[src*="hero"]` matches `/uploads/2025/hero.jpg`, `hero-mobile.jpg` and any other image whose URL contains the word 'hero'.

Class selectors — most precise

Target elements by an explicit class name. This is the most precise option but requires you to put the class on the element, which the visual editor does not do for arbitrary classes. The workflow is:

- Write the rule in the CSS drawer: `.card { padding: 20px; border-radius: 8px; }`
- Open the editor's source view: View → Source code, or the `</>` button on the toolbar.
- Find the element you want to style and add the class to it: change `<div>` to `<div class="card">`.
- Click Save to push the change back into the editor. The class takes effect in the editor preview immediately and appears in the output.

Source view feels like a small detour the first few times. It is also the only way to assign arbitrary classes from a visual editor, short of building a custom class picker into the editor itself.

Pseudo-classes — interactive states

Hover, focus, first-child and friends all work. These are mostly only visible in the output's live use, not in the editor preview — the editor's body has its own interaction layer over the top — but they still get included in the generated CSS for use on the receiving page:

```
a {
  color: #2a5cad;
  text-decoration: underline;
}
a:hover {
  text-decoration: none;
  background: #eef3fb;
}
p:first-of-type::first-letter {
  font-size: 3em;
  font-weight: bold;
  float: left;
  line-height: 1;
  margin: 4px 8px 0 0;
}
```

9. Worked example: a styled hero section

Let us build a small landing-page hero from scratch, using the editor for the content structure and the Custom CSS drawer for the styling. This example uses tag selectors and one class, exercises the live preview, and shows the resulting output.

Step 1: compose the content

In the editor, type the following — use the Heading 1 format for the headline, and the Paragraph format for the supporting text:

Build websites worth keeping.

Honest WordPress development from a one-person studio.
No upsells, no premium tiers, just plugins that work.

Get started

Insert an image above the headline using the Image button — pick any URL from your media library.

Step 2: open the CSS drawer and write the styles

Click the CSS button on the editor's panel header. The drawer slides up and the textarea takes focus. Paste in:

```
body {
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif;
  color: #1f2330;
  line-height: 1.6;
}

h1 {
  font-size: 48px;
  line-height: 1.1;
  letter-spacing: -0.02em;
  margin: 0 0 16px 0;
}

p {
  font-size: 18px;
  color: #4a5060;
  max-width: 540px;
  margin: 0 0 24px 0;
}

img { border-radius: 12px; margin-bottom: 24px; }

/* Style the call-to-action paragraph. We will add the class
   to it in step 3 via Source code view. */
```

```
.cta {
  display: inline-block;
  padding: 12px 24px;
  background: #2a5cad;
  color: #ffffff;
  border-radius: 8px;
  text-decoration: none;
  font-weight: 600;
}
```

As you paste, the editor on the left transforms in real time: the heading goes large and tight, the supporting text widens and goes grey, the image gets rounded corners. The 'Get started' paragraph is still unstyled because we have not yet told it which class to wear.

Step 3: apply the .cta class via source view

Open View → Source code. Find the paragraph containing 'Get started' and change it from

```
<p>Get started</p>
```

to

```
<p class="cta">Get started</p>
```

Click Save. The modal closes and the 'Get started' paragraph instantly turns into a styled button in the editor preview.

Step 4: copy the output

The output panel on the right has been updating throughout. With Tidy on (default) and Classify off, the output is your CSS plus the editor's clean HTML, ready to paste anywhere CSS will be applied:

```
<style>
body { ... }
h1 { font-size: 48px; ... }
...
.cta { padding: 12px 24px; ... }
</style>

<h1>Build websites worth keeping.</h1>
<p>Honest WordPress development...</p>
<p class="cta">Get started</p>
```

Click Copy and paste into your destination. Done.

10. Worked example: refactoring a TinyMCE document with Classify

This time, we start from a heavily-styled document built using TinyMCE's colour and size pickers — the visual route, no CSS knowledge required — and use Classify to refactor the result into shareable CSS classes.

Step 1: compose visually

In the editor, build a small newsletter heading. Use TinyMCE's text colour picker to set headings red, the font-size dropdown to make them large, and the background-colour picker to highlight some text. Mix in a couple of styled paragraphs. You are working visually; the underlying HTML is being filled with inline style attributes.

With Tidy on and Classify off, the output panel shows something like:

```
<h1 style="color: #e03e2c; font-size: 36px;">November Newsletter</h1>
<p style="color: #4a5060;">Welcome back. Here is what we have been up to.</p>
<h2 style="color: #e03e2c; font-size: 24px;">New plugin release</h2>
<p style="color: #4a5060;">
  We shipped <span style="background-color: #fff3a8;">version 1.7.1</span> last week.
</p>
<h2 style="color: #e03e2c; font-size: 24px;">Coming up</h2>
<p style="color: #4a5060;">Two more plugin updates next month.</p>
```

Functional, but bloated. Three repeated colour declarations, two repeated font-size declarations, and the file is going to grow proportionally with every new section.

Step 2: turn on Classify

Click the Classify button in the output panel header. The output transforms:

```
<style>
.c-e03e2c { color: #e03e2c; }
.fs-36 { font-size: 36px; }
.c-4a5060 { color: #4a5060; }
.fs-24 { font-size: 24px; }
.bg-fff3a8 { background-color: #fff3a8; }
</style>
<h1 class="c-e03e2c fs-36">November Newsletter</h1>
<p class="c-4a5060">Welcome back. Here is what...</p>
<h2 class="c-e03e2c fs-24">New plugin release</h2>
<p class="c-4a5060">
  We shipped <span class="bg-fff3a8">version 1.7.1</span> last week.
</p>
<h2 class="c-e03e2c fs-24">Coming up</h2>
<p class="c-4a5060">Two more plugin updates next month.</p>
```

Five rules in the <style> block — one for every unique declaration — and the duplicates across the body have all collapsed onto shared classes. Add another color:

#e03e2c element later and no new rule appears.

Step 3: optionally add base styles via the drawer

Classify has handled the colours and sizes. To layer on typography and spacing, open the CSS drawer and add:

```
body { font-family: Georgia, serif; line-height: 1.7; }
h1 { margin: 0 0 8px 0; }
h2 { margin: 32px 0 8px 0; }
p { margin: 0 0 16px 0; }
```

These tag-selector rules merge into the same <style> block as Classify's output, sitting above the atomic rules:

```
<style>
body { font-family: Georgia, serif; line-height: 1.7; }
h1 { margin: 0 0 8px 0; }
h2 { margin: 32px 0 8px 0; }
p { margin: 0 0 16px 0; }
.c-e03e2c { color: #e03e2c; }
...rest of atomic rules...
</style>
...body...
```

The result is a small, self-contained piece of HTML+CSS that you can drop into an email, a CMS, a static page, or anywhere else that supports inline <style> blocks.

11. Combining Tidy, Classify and Custom CSS

All three stages are toggleable independently, giving eight possible combinations. In practice four are useful, three are occasionally useful, and one is the default. A cheat-sheet:

Tidy	Classify	Custom CSS	When to use it
On	Off	Empty	The default. Clean semantic markup with inline styles preserved. Good for pasting into systems that already have a stylesheet.
On	Off	Has content	Compose with the editor, style with the CSS drawer. The output is your CSS plus clean HTML. Best for one-off content where you control both.
On	On	Empty	Inline styles get refactored into shared atomic classes. Best for visually-composed content with repeated styles that you want to consolidate.
On	On	Has content	The fullest treatment. Your base styles plus auto-generated atomic utilities, both in one <code><style></code> block. Best for repeated newsletters, marketing emails, content templates.
Off	—	—	Raw TinyMCE output. Useful only for debugging or when you specifically want to see what TinyMCE produced before any cleanup.

The pipeline order

When more than one stage is active, they run in this order:

- **Tidy** first, on the raw TinyMCE HTML. Cleans up spans, unwraps standalone images, drops empty paragraphs.
- **Classify** next, on Tidy's output. Extracts inline styles into atomic classes and emits the rules as a `<style>` block.
- **Custom CSS injection** last. If the drawer has content, it either merges into Classify's `<style>` block (as the first rules) or emits its own block at the top if Classify was off.

This order is fixed and not configurable. The reasoning: Tidy first gives Classify cleaner input to work with; Custom CSS last lets your CSS sit at the top of the output where it is most discoverable, and lets it act as the base layer for the atomic utilities below.

12. Tips, tricks and idioms

Use the editor as a styled scratchpad

Even without copying the output anywhere, the editor + CSS drawer combination is useful as a scratchpad for prototyping a piece of content. Quick way to test what a card layout will look like, whether two colours work next to each other, how a heading sits next to a paragraph. The live preview shortens the feedback loop dramatically compared to editing CSS in a real codebase.

Combine attribute selectors with classes

For images in particular, attribute selectors save you a trip into source view. Want different border radii for hero shots versus inline images? Set alt text on the hero shot via TinyMCE's Image dialog and write `img[alt="Hero"] { ... }`. The hero alt is also good for accessibility, so you get two wins for one input.

Use the Format dropdown for common block patterns

TinyMCE's block format dropdown can switch a paragraph into a blockquote, a preformatted block, or any heading level. Combined with tag-selector CSS in the drawer (`blockquote { ... }`), this gives you styled blockquotes without ever assigning a class.

CSS variables work fine

Define your palette once and reuse it throughout the document:

```
:root {
  --brand: #2a5cad;
  --ink: #1f2330;
  --soft: #4a5060;
}

h1 { color: var(--brand); }
p { color: var(--soft); }
a { color: var(--brand); }
a:hover { color: var(--ink); }
```

Change one variable, every consumer updates. CSS variables travel with the output, so the receiving page picks them up the same way.

Media queries are fine too

If your output is going somewhere responsive, write media queries in the drawer. They will not affect the editor (which is a fixed-width iframe) but they will appear in the output and take effect on the live page:

```
h1 { font-size: 32px; }
```

```
@media (min-width: 768px) {  
  h1 { font-size: 48px; }  
}  
  
@media (min-width: 1200px) {  
  h1 { font-size: 64px; }  
}
```

Keep Tidy on; toggle Classify per-job

There is rarely a reason to turn Tidy off. Classify, on the other hand, is useful for some workflows and noise for others. Configure your defaults as Tidy on, Classify off, and only flip Classify on when you specifically want the inline-styles-to-classes refactor. The toggles are remembered across sessions so set them once and forget.

13. Privacy and local storage

The plugin stores nothing on the server beyond its own activation state. Your editor content, your CSS, your toggle preferences and your theme choice all live exclusively in your browser's local storage. Nothing is transmitted, logged, or shared with any third party.

The keys, all under the `mbrHtmlGen.` namespace:

Key	Value	What it holds
<code>mbrHtmlGen.content</code>	string	The editor's current contents. Updated half a second after you stop typing.
<code>mbrHtmlGen.format</code>	1 or 0	Pretty-print toggle state.
<code>mbrHtmlGen.wrap</code>	1 or 0	Soft-wrap toggle state.
<code>mbrHtmlGen.tidy</code>	1 or 0	Tidy toggle state.
<code>mbrHtmlGen.classify</code>	1 or 0	Classify toggle state.
<code>mbrHtmlGen.customCss</code>	string	The Custom CSS drawer's contents.
<code>mbrHtmlGen.cssDrawerOpen</code>	1 or 0	Whether the CSS drawer is open.
<code>mbrHtmlGen.theme</code>	light/dark	Theme choice.

Clearing local storage

To start completely fresh: in your browser's developer tools, open Application (Chrome/Edge) or Storage (Firefox), find Local Storage, expand your site, and delete the `mbrHtmlGen.` keys. Or clear all site data for your domain, which removes them along with other site state.

Multiple browsers, multiple states

Because the state is browser-local, the plugin presents a different remembered state in Chrome versus Firefox, and on your laptop versus your phone. This is usually fine — most people use one browser per device — but it means you cannot compose a document on one machine and pick it up on another without copying the output across.

14. The frontend shortcode

Everything covered so far works identically on the frontend via the `[mbr_html_generator]` shortcode. Drop it into any page or post and the same tool renders inline, with the same toolbars, the same CSS drawer, the same toggle states. Visitors who are not logged in can use the tool the same way you can.

Why you might use the shortcode

- **Public demos.** Show readers what your tool produces without making them log in.
- **Documentation pages.** Pair a 'try it yourself' embed with explanatory text.
- **Lead generation.** Free utility on your site is a friendly first touchpoint with potential clients.
- **Internal team tools.** A page in your members area that gives the team easy access to a compose-style-copy workflow.

Things to know

The shortcode loads its own copy of TinyMCE and the rest of the plugin's assets, scoped to the shortcode container. Multiple shortcodes on the same page are supported but rare — the first instance takes precedence and subsequent ones show a notice. Frontend visitors see the same theme toggle but the default theme follows their operating-system preference (light or dark) rather than your admin choice.

Like the admin tool, the shortcode runs entirely client-side. No data leaves the visitor's browser. The visitor's CSS, content and toggles persist in their browser's local storage just like yours do in the admin.

15. Troubleshooting

The CSS I wrote does not appear in the editor preview

Three things to check. First, give it 200 milliseconds — the live preview is debounced. Second, check that the selector actually matches something in the editor; a rule like `.card` has no effect unless an element in the document has `class="card"` on it. Third, check the browser console for CSS parsing errors — a single misplaced bracket can throw off everything that follows.

My CSS appears but is being overridden

Most likely a specificity issue. TinyMCE applies its own baseline styles to the content iframe (a comfortable body padding, a default font, sensible link colours) via its `content_style` configuration. These have low specificity but they are loaded before your CSS, so equally-specific rules win for TinyMCE. Either bump your selector's specificity (use a class or an attribute selector instead of a bare tag), or add `!important` for the few rules you really need to enforce.

Source view is empty or stuck

If View → Source code opens an empty modal or hangs, your editor content may have hit an edge case in TinyMCE's source serialiser. Try clicking Cancel to dismiss the modal, then making a small edit in the visual editor (type and delete a character) to nudge the internal state. If the problem persists, copy the editor content somewhere safe and clear it from the editor; usually a single problematic node is the cause and identifying it from a clean slate is easier.

Classify produced too many counter-based classes

Classes like `c-1`, `bg-2`, `ff-1` indicate values that contain characters Classify cannot safely put in a class name: `rgb()`, `rgba()`, `url()`, `calc()`, `var()`, quoted strings, or comma-separated lists. The classes work fine but are unreadable. Two options: switch to hex colours via TinyMCE's colour picker (which produces values Classify can name), or rewrite any `var()` calls in your custom CSS to hard-coded values for the final output.

The output panel says one thing, the clipboard has another

Should not happen — Copy uses exactly the same pipeline as the output panel. If you are seeing a mismatch, the most likely cause is that you copied while a debounced save or render was mid-flight; clicking Copy again typically resolves it. If the mismatch persists, please report it via the GitHub repository with the editor content and toggle state attached.

Older browsers

The plugin uses `DOMParser`, `Array.prototype.indexOf`, `Element.classList` and a handful of other APIs from the IE9+ era. Realistically anything from the last decade works. Internet Explorer 11 will load the page but fail to render the editor; the rest of the plugin will work as a static display.

MBR HTML Generator is built and maintained by Robert Palmer at Made by Robert (madebyrobert.co.uk) and Little Web Shack (littlewebshack.com). Free, open-source, and never updates itself via WordPress.org. To get notified about new versions, follow the [GitHub repository](#) or sign up to the [Little Web Shack newsletter](#).